
Programovací jazyky a překladače

Přednáška 8, poznámky

Jan Voráček

VŠPJ Jihlava, voracek@vspj.cz

Administrativa

- Byly vypsány zkouškové termíny
 - V Průvodci si zopakujte podmínky pro získání zápočtu a absolvování zkoušky
 - Předpoklady pro udělení zápočtů k předtermínům budou vyhodnocovány nejpozději k těmto datům:
 - Pro termín 22.5. musí být veškeré náležitosti dostupné v Moodle nejpozději 18.5., 23:59
 - Pro termín 24.5. musí být veškeré náležitosti dostupné v Moodle nejpozději 20.5., 23:59
 - Po zveřejnění složení komisí pro SZZ bude doplněn jeden termín ke konci června
- Prezentace
 - YACC, sémantická analýza
 - Syntaxí řízený překlad
 - Atributové gramatiky

Dnešní témata

- Zopakování funkce tabulkového analyzátoru LL(1) jazyků
- Teoretické základy překlady LR gramatik
 - Precedenční gramatiky
 - LR překladové tabulky
- Úvod do sémantické analýzy
 - Kromě analýzy kontextu v ní jde hlavně o akce, které překladač generuje
 - Zde se už naplno uplatní i údaje, uložené v tabulce symbolů
- Generátory syntaktických analyzátorů: *yacc/Bison*
 - Sémantická pravidla v akcích (proměnné typu \$)

Precedenční gramatiky, definice

- Jsou typu LR a jejich terminály mají tabulkou přiřazenou prioritu a asociativitu
- Na jejich základě se hledají redukovatelné sekvence (handles) v zásobníku

$+ < \bullet *$
 $* \bullet > +$ } $+$ má nižší **prioritu** než $*$

$+ \bullet > +$
 $* \bullet > *$ } $+$ a $*$ jsou zleva **asociativní**

	id	+	*	\$
id		$\bullet >$	$\bullet >$	$\bullet >$
+	$< \bullet$	$\bullet >$	$< \bullet$	$\bullet >$
*	$< \bullet$	$\bullet >$	$\bullet >$	$\bullet >$
\$	$< \bullet$	$< \bullet$	$< \bullet$	

- Omezení precedenčních gramatik:
 - **Neobsahují ϵ pravidla a dva sousedící neterminály**
- Vstupní symboly umísťujeme na zásobník zároveň s jejich prioritami a asociativitami.
- Počáteční stav zásobníku je $\$$
- V zásobníku **hledáme úseky** se stejnou prioritou $< \bullet \dots \bullet >$, které ve shodě s gramatickými pravidly nahrazujeme odpovídajícími neterminály. Každá redukce je doprovázena příslušnou akcí. Existuje celá řada notací, kterými lze tuto činnost znázornit.

Využití precedenční grammatiky k LR(1) překladu

$E ::= E + E \mid E * E \mid id$

	+	*	id	\$
+	>	<	<	>
*	>	>	<	>
id	>	>		>
\$	<	<	<	>

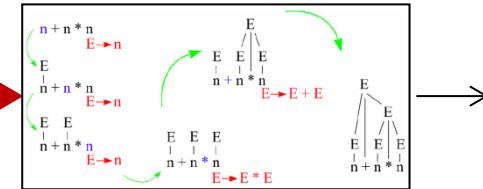
Stack	Input	Precedence
\$	id + id * id \$	\$ < id
\$ < id	+ id * id \$	id > +
\$ < E	+ id * id \$	\$ < +
\$ < E +	id * id \$	+ < id
\$ < E + < id	* id \$	id > *
\$ < E + < E	* id \$	+ < *
\$ < E + < E *	id \$	* < id
\$ < E + < E * < id	\$	id > \$
\$ < E + < E * E	\$	* > \$
\$ < E + E	\$	+ > \$
\$ < E	\$	\$ > \$

AKCE

Analýza zdola nahoru

- LR(1) gramatiky:
 - Čte vstupní větu zleva doprava (L)
 - Provádí její (reverzní) pravý (= nejpravější) rozklad (R)
 - Deterministická predikace dalšího kroku na základě znalosti následujícího 1 vstupního tokenu (1)

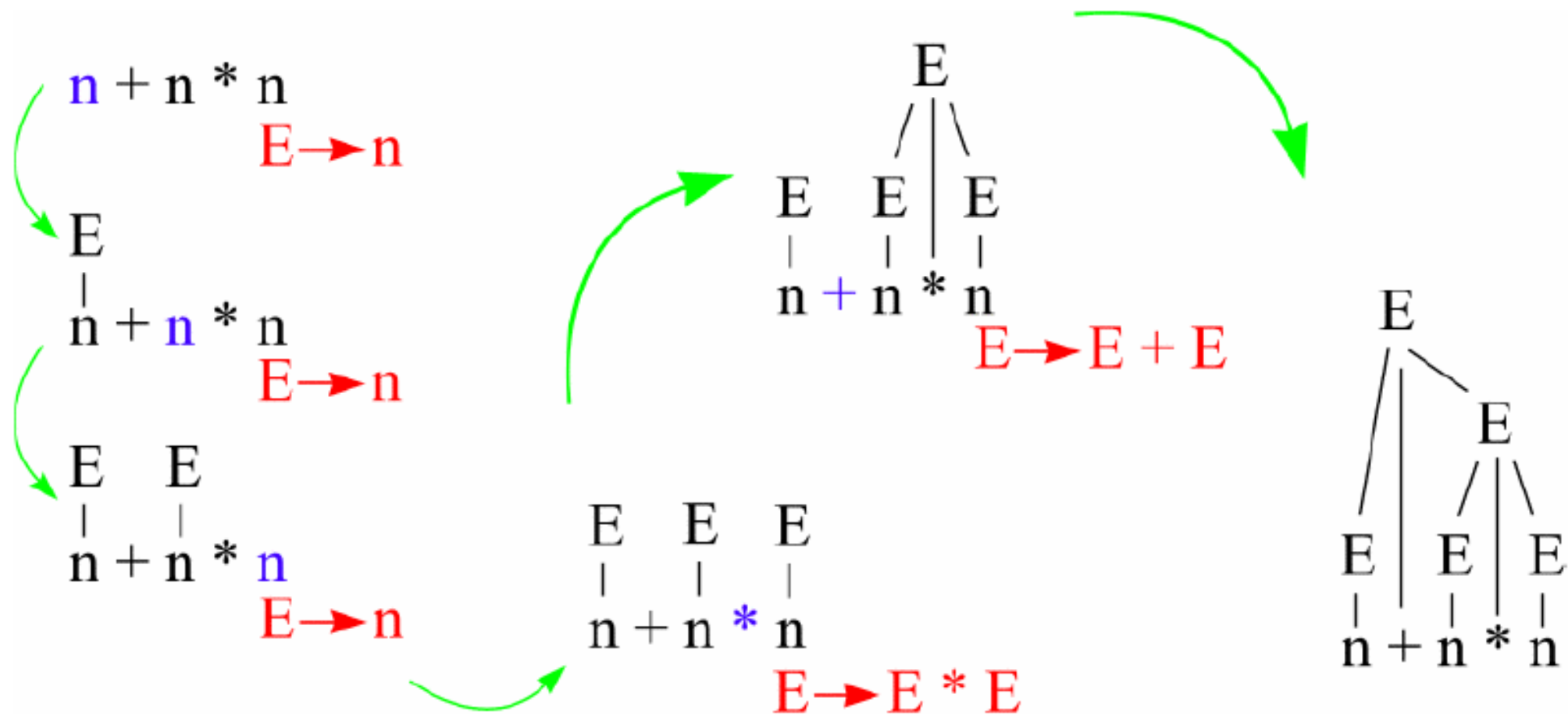
Prováděný rozklad (výstavba stromu)



$\underline{E} \Rightarrow E + \underline{I} \Rightarrow E + T * \underline{E} \Rightarrow E + \underline{I} * n \Rightarrow$
 $\Rightarrow E + \underline{F} * n \Rightarrow \underline{E} + n * n \Rightarrow \underline{I} + n * n \Rightarrow$
 $\Rightarrow \underline{F} + n * n \Rightarrow n + n * n$

Reverzní směr zpracování vstupu (průběžné redukce)

Výstavba derivačního stromu zdola



Analýza LR(1) jazyků

Miroslav Beneš

Dušan Kolář



- 1) $E \rightarrow E+T$
- 2) $E \rightarrow T$
- 3) $T \rightarrow T * F$
- 4) $T \rightarrow F$
- 5) $F \rightarrow (E)$
- 6) $F \rightarrow id$

state	id	+	*	(	)	\$		E	T	F
0	s5			s4				1	2	3
1		s6				acc				
2		r2	s7		r2	r2				
3		r4	r4		r4	r4				
4	s5			s4				8	2	3
5		r6	r6		r6	r6				
6	s5			s4					9	3
7	s5			s4						10
8		s6			s11					
9		r1	s7		r1	r1				
10		r3	r3		r3	r3				
11		r5	r5		r5	r5				

• Zásobník

						Vstup	Akce
					0	n+n*n	s5
					0	n,5	+n*n
					0	F,3	r F -> n
					0	T,2	+n*n
					0	E,1	r T -> F
					0	E,1	+n*n
					0	E,1	r E -> T
					0	E,1	+n*n
					0	E,1	s6
					0	E,1	+n*n
					0	E,1	s5
					0	E,1	*n
					0	E,1	r F -> n
					0	E,1	+n
					0	E,1	*n
					0	E,1	r T -> F
					0	E,1	+n
					0	E,1	s7
					0	E,1	*n
					0	E,1	s5
					0	E,1	+n
					0	E,1	*n
					0	E,1	r T -> F
					0	E,1	+n
					0	E,1	*n
					0	E,1	r T -> T * F
					0	E,1	+n
					0	E,1	r E -> E + T
					0	E,1	acc

LR gramatika a její překladová tabulka

$S \rightarrow a A S$ (1)

$S \rightarrow b A$ (2)

$A \rightarrow c A$ (3)

$A \rightarrow d$ (4)

	a	b	c	d	\$	S	A
0	S2	S3				1	
1					OK		
2			S5	S6			4
3				S6			7
4	S2	S3				8	
5			S5	S6			9
6		R4			R4		
7					R2		
8					R1		
9		R3			R3		

LR syntaktická analýza vstupu **acdbd**, řešení

Zásobník	Vstup	Akce
0	acdbd \$	S2
0a2	cdbd\$	S5
0a2c5	dbd\$	S6
0a2c5d6	bd\$	R4
0a2c5A9	bd\$	R3
0a2A4	bd\$	S2
0a2A4b3	d\$	S6
0a2A4b3d6	\$	R4
0a2A4b3A7	\$	R2
0a2A4S8	\$	R1
0S1	\$	Accept

$S \rightarrow a A S$ (1)

$S \rightarrow b A$ (2)

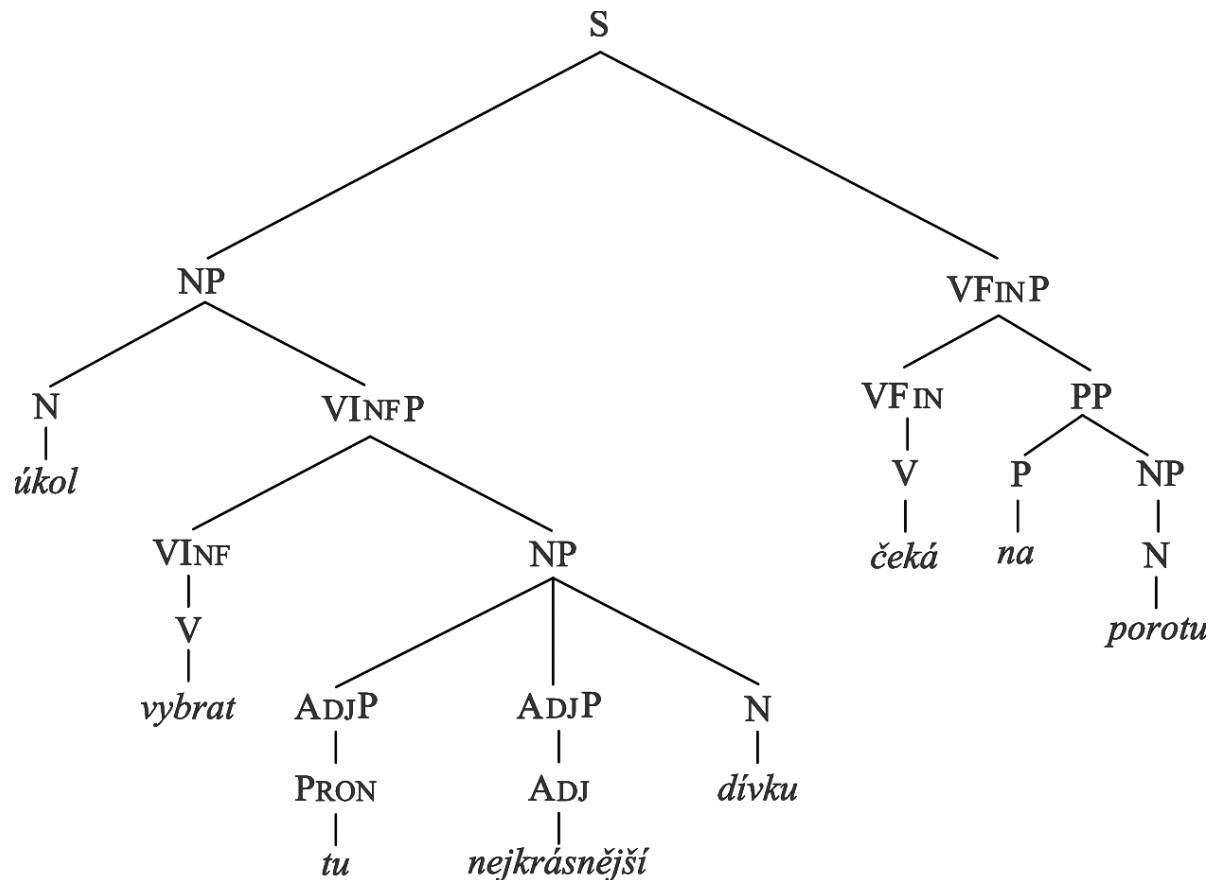
$A \rightarrow c A$ (3)

$A \rightarrow d$ (4)

	a	b	c	d	\$	S	A
0	S2	S3				1	
1					OK		
2			S5	S6			4
3				S6			7
4	S2	S3				8	
5			S5	S6			9
6		R4			R4		
7					R2		
8					R1		
9		R3			R3		

Syntaxe: obecná větná struktura/šablona korektních vět jazyka

Sémantika: konkrétní význam syntakticky korektních vět jazyka



Adj - přídavné jméno (adjektivum)

AdjP - adjektivní skupina

N – podstatné jméno (substantivum)

NP – jmenná skupina

P – předložka

PP – předložková skupina

Pron – zájmeno

S – věta

V – sloveso,

VFin – určité (finitní) sloveso

VFinP – slovesná skupina řízená finitním slovesem

VInf – infinitiv

VInfP – slovesná skupina řízená infinitivem

Zdroj: <https://www.czechency.org/slovník/PARSING>

Kde syntaxe nestačí

- Je x skalár, pole nebo funkce? Bylo x deklarováno? Pokud ano, bylo použito? Vyžaduje-li definici, byla před použitím provedena?
- Existuje v jednom programu více x ?
- Je výraz $x * y + z$ typově konzistentní?
- Bylo-li a deklarováno jako $a[i, j, k]$, jsou mu při použití přiřazeny hodnoty všech dimezí?
- Kam uložíme proměnnou z ?
 - *Např. do registru, lokálně, globálně, staticky nebo na heap*
- Kolik a jakého typu jsou argumenty funkce $fie()$?
- Odkazuje $*p$ na výsledek, vrácený funkcí typu $malloc()$?
- Odkazují p a q na stejné adresy?

Sémantika, tj. kontextové informace v C

```
fie(a, b, c, d) ←  
    int a, b, c, d;  
{ ... }
```

```
fee()  
{  
    int f[3], g[0], h, i, j, k; 2  
    char *p;  
    call fie(h, i, "ab", j, k); ←  
    k = f * i + j;  
    h = g[17];  
    printf("<%s,%s>.\n", p, q);  
    p = 10;  
}
```

V čem je problém:

g[i]

argumenty *fie()*

typ "*ab*"

dimenze *f*

nedeklarovaná proměnná *q*

10 není řetězec

Rozsahová úskalí následujícího kódu

```
int Foo() {  
    ↪ int x = 137;  
    {  
        string x = "Ahoj!"  
        if (float x = 0)  
            double x = x;  
    }  
    if (x == 137) cout << "OK";  
}
```

Gramatická struktura programu

Program --> D S E

Deklarace (Declarations, D)

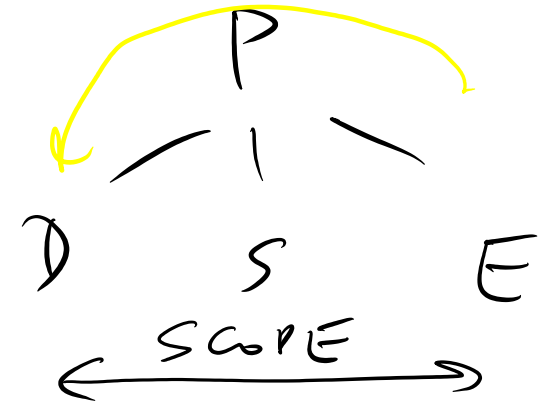
- Sémantika (typy, parametry, rozsah platnosti)
- Žádný kód

Příkazy (Statements, S)

- Řízení chodu programu (labels, goto, jump, exit, break)

Výrazy (Expressions, E)

- Generování výkonných instrukcí, modifikujících paměťová místa



Chování překladače

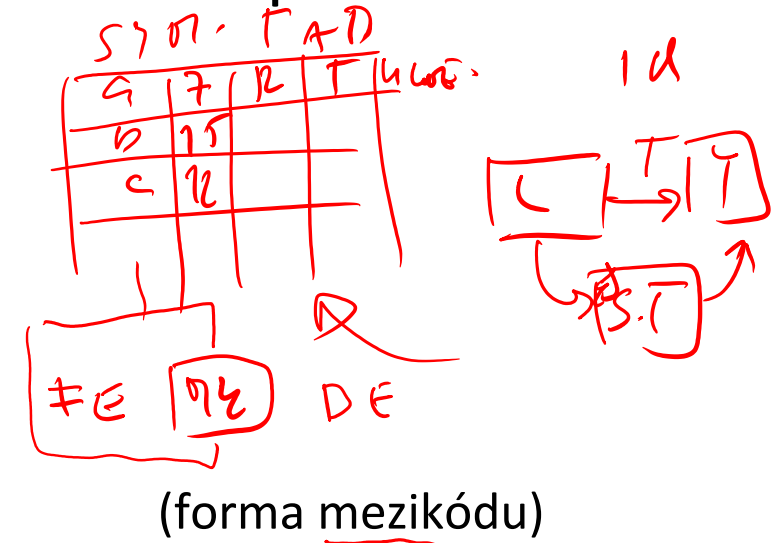
Překladač není jen syntaktický analyzátor, ale po spuštění a zpracování zdrojového programu musí generovat výstup:

1. Přímo realizovaný

- Kalkulátor aritmetických výrazů
 - Vstup: $1 + 2 * 3$ nebo $a + b * c$,
- Převod to postfixové notace
 - Vstup: $1 + 2 * 3$,

výstup: 7

výstup: $1\ 2\ 3\ *\ +$



2. Následně spustitelný na cílové platformě

- Sada instrukcí pro cílový procesor
 - Fyzický
 - Virtuální
 - **Mezikód**

Syntaxí řízený překlad

Sémantické (významové, výstupní) akce **synchronní se syntaktickou kontrolou**

- Zajišťují jednopřechodovost překladu
- Eliminují výrazové nedostatky bezkontextových gramatik
- Přiřazují hodnoty a další atributy pojmenovaným terminálním i *neterminálním* symbolům
 - Rozsah platnosti (scope)
 - Typová kompatibilita a další
 - Informace se průběžně ukládají do tabulky symbolů

Syntaxí řízený překlad

Miroslav Beneš
Dušan Kolář



Sémantické akce a zásobníky v yacc_u

- **Syntaktický** (shift, reduce) – tokeny, vrácené *lex_em*
- **Sémantický** (hodnoty, \$i) – hodnoty proměnné *yylval* z *lex_u* založené na obsahu proměnné *yytext*

Spolupráce obou zásobníků:

yyparse -> yylex -> *yylval* -> (shift do sémantického zásobníku)
-> {akce s hodnotami \$1, \$2, ..} -> (redukce pravidla)
-> získání hodnoty \$\$ -> aktualizace zásobníku

Význam sémantických proměnných:

- **\$\$**: levá strana aktuálního pravidla
- **\$i**: člen pravé strany aktuálního pravidla, daný svým pořadovým číslem
 - Stejně se počítají i zapouzdřené sémantické akce
 - Implicitní akce: $$$ = \1
- **\$<typ>0**: první gramatický symbol s dědičným atributem (předešlé hodnoty pravých stran předcházejících pravidel v zásobníku). K případným dalším atributům se dostaneme dekrementací.

Sémantické akce 1

Syntetizované (*synthesized*) atributy

expression: expression '+' expression { \$\$ = \$1+\$3; }

| expression '-' expression { \$\$ = \$1-\$3; }

| expression '*' expression { \$\$ = \$1*\$3; }

| expression '/' expression { \$\$ = \$1/\$3; }

YACC a sémantika, složitější příklad

S/R : SNIFF
R/N : DONT'VE TO PLAIN

```
/* variables */
[a-z] {
    yyval = *yytext - 'a';
    return VARIABLE;
}

/* integers */
[0-9]+ {
    yyval = atoi(yytext);
    return INTEGER;
}
```

LEX

```
%token INTEGER VARIABLE
%left '+' '-'
%left '*' '/'

%{
    void yyerror(char *);
    int yylex(void);
    int sym[26];
}%
```

```
program:
    program statement '\n'
    | ε
    ;
```

```
statement:
    expr
    | VARIABLE '=' expr
    ;
```

```
expr:
    INTEGER
    | VARIABLE
    | expr '+' expr
    | expr '-' expr
    | expr '*' expr
    | expr '/' expr
    | '(' expr ')'
    ;
```

```
%}

void yyerror(char *s) {
    fprintf(stderr, "%s\n", s);
    return 0;
}

int main(void) {
    yyparse();
    return 0;
}
```

MAIN

3+4 → 7

x=3

y=7+2

x → 3

y → 9

x+y → 12

SNIFF

x=3

merge

USPIS

```
{ printf("%d\n", $1); }
{ sym[$1] = $3; }
```

USPIS

```
{ $$ = sym[$1]; }
{ $$ = $1 + $3; }
{ $$ = $1 - $3; }
{ $$ = $1 * $3; }
{ $$ = $1 / $3; }
{ $$ = $2; }
```

Sémantické akce 2

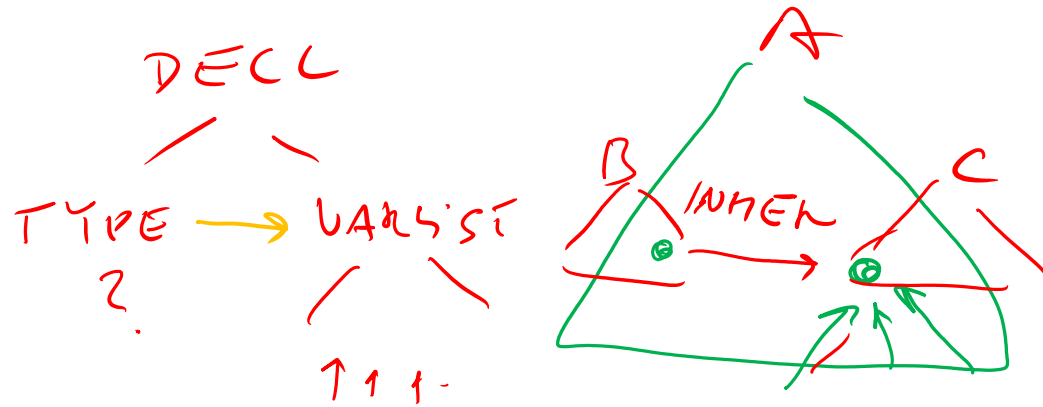
Dědičné (*inherited*) atributy

YACC, pravidla a akce :

```
decl: type varlist
$0 type: INT | FLOAT
varlist:
    VAR
    | varlist ',' VAR
```

Analýza:

```
. INT VAR
INT . VAR
type . VAR
type VAR .
type varlist .
decl .
```



```
{ setType($1, $0); }
{ setType($3, $0); }
```

int a, b, c, d...

}

V okamžiku této redukce potřebuje yacc instalovat typ proměnné do tabulky symbolů. Ten ale není dostupný v rámci aktuálního pravidla, tj. na vrcholu sémantického zásobníku.

Další sémantické poznámky

U dědičných atributů ($\$i$ pro $i \leq 0$) je vhodné uvádět i jejich typ, tj. $\$<typ>0$.

Je-li $\$i$ neterminál (v *yacc_u* je obvykle značíme malými písmeny), pak je přiřazená hodnota převzata z akce nejbližšího dříve aplikovaného pravidla. Je-li $\$i$ terminál ($\%token$), je jeho hodnota převzata z lexikálního analyzátoru (*yy1val*).

Levá rekurze by v gramatice měla dostávat přednost před pravou rekurzí. Důvodem je především výsledná hloubka zásobníku:

